

TEAM GUIDE · READ BEFORE YOUR FIRST TASK

Alarsio

Team Development Guide

Four people, eight weeks, nine modules. This guide is how we turn that into a working product without stepping on each other. It covers the loop we follow for every feature, how to get real work out of Claude Code, and the rules that keep four parallel streams from colliding.

4

people
two modules each

9

MVP modules
nothing more

8

weeks
foundation, capture, close

1

repo
monorepo + Turborepo

Contents

Day one — get running	02
How we work — the loop	03
The ten steps, in detail	04
Claude Code — the essentials	05
Claude Code — power features	06
Our commands, subagents and skills	07
The documents that run the project	08
Git, PRs and review	09
Testing and security review	10
CI/CD with GitHub Actions	11
Who builds what, week by week	12

Day One — Get Running

Under an hour, in order. The git identity step is the one that silently costs you marks if you skip it.

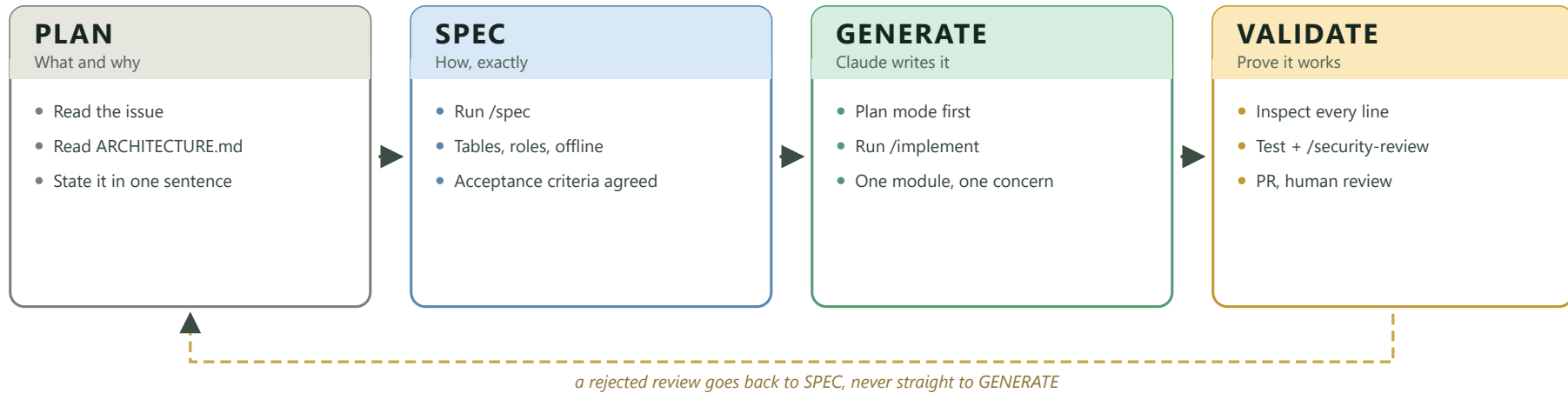
Step	What to do
Accounts	GitHub with 2FA on · accept the alarsio-labs invite · Supabase free tier · Claude Code installed and signed in
Tools	Node 20+ · pnpm 9+ · Git · Docker Desktop · GitHub CLI
Git identity critical	<code>git config --global user.email</code> must match an email on your GitHub account. Wrong email means your commits never appear on your profile or the contribution graph — and the programme is graded on that.
Clone and install	<code>gh auth login</code> · <code>git clone</code> · <code>pnpm install</code> · copy <code>.env.example</code> to <code>apps/web/.env.local</code> and fill in your Supabase values
Never commit	The service-role key. Not once, not temporarily. CI fails the build if the string appears anywhere in the repo.
Read, in order	<code>CLAUDE.md</code> → <code>docs/WORKFLOW.md</code> → <code>docs/ARCHITECTURE.md</code> → <code>docs/FEATURES.md</code> → <code>docs/IMPROVEMENTS.md</code>
First contribution today	Add your name to the README on a branch, open a PR, have someone else merge it. Learn the loop before it matters.

Your first task today is a throwaway PR — add your name to the README, open a PR, have someone else merge it. Learn the loop on something that does not matter, before doing it on something that does.

How We Work — The Loop

Every feature goes through the same four phases. Nothing is generated before a spec exists.

ONE FEATURE, ONE PASS THROUGH THE LOOP



The spec is what makes generated code reviewable. Without one, nobody can say whether the output is right — only whether it looks plausible. That difference is the whole reason we work this way.

The Ten Steps, In Detail

This is the loop expanded. Steps 4 and 9 are the ones people skip, and they are the two that decide whether the code is any good.

#	Step	What it means
1	Understand the requirement	Read the issue, the module's section in ARCHITECTURE.md, and the P0 rows in PRD.md. You are done when you can state in one sentence what the user can now do, and which module owns the data. If you cannot, the issue is not ready — ask, don't guess.
2	Design and think	Before Claude is involved, write in the issue: which tables change and who owns them, which module functions you will call and which you must not, what the user sees when it works and when it fails, what breaks offline. Five minutes here saves an hour.
3	Ask Claude Code to implement	Start in plan mode. Give the spec, not a vague request. Review the plan before approving it — if the plan names a table your module does not own, stop there. That is far cheaper than reviewing the diff.
4	Inspect the generated code	Read every line. This is the step people skip and the step that matters. Does it do only what the spec said? Does it touch another module's tables? Any <code>any</code> , swallowed error or TODO? If you cannot explain a line to a teammate, do not ship it.
5	Test	<code>pnpm typecheck</code> , <code>lint</code> , <code>test</code> , and <code>test:db</code> if SQL changed. Then by hand, including the unhappy path: no network, bad input, wrong role.
6	Security review	Run <code>/security-review</code> , then walk <code>docs/SECURITY.md</code> yourself. The automated pass finds the obvious; the checklist finds the rest. Never skip it on auth, RLS, worker data or money.
7	Commit	Small, logical commits. <code>git add</code> specific files — read what you are staging. Imperative subject: “Add geofence check to site check-in”.
8	Open the PR	<code>gh pr create --fill</code> , then fill the template honestly. “What I checked” is the section that tells your reviewer where to look.
9	Human review	A different person reviews — never the module owner. That is how knowledge spreads across four people instead of pooling in one. “Looks good” without reading is worse than no review.
10	Merge and update the record	CI green, then merge. Tick the row in FEATURES.md. If you made a non-obvious choice, add an ADR to DECISIONS.md.

Claude Code — The Essentials

Learn these seven before anything else. They cover most of what you will do day to day.

Feature	How	Why it matters
Plan mode	<code>Shift+Tab</code>	Claude plans before touching anything. Use it for every task bigger than a one-line fix. Reviewing a plan costs seconds; reviewing a wrong diff costs an hour.
Project memory	<code>CLAUDE.md</code>	Read automatically at the start of every session. Our rules, stack, module owners and conventions live there. Keep it short and true — a stale CLAUDE.md actively misleads.
Clear the context	<code>/clear</code>	Start a fresh task with a clean slate. A long session drifts; old context makes Claude confident about things that changed twenty messages ago.
Security review	<code>/security-review</code>	Reviews the pending diff for vulnerabilities. Mandatory on anything touching auth, RLS, worker data or money.
Code review	<code>/code-review</code>	Reviews the current diff for correctness bugs before a human sees it.
Install the GitHub app	<code>/install-github-app</code>	Sets up the Claude GitHub Action and its secret. Run once, by a repo admin. After that, mention @claude in any issue or PR.
Initialise memory	<code>/init</code>	Generates a first CLAUDE.md by reading the codebase. Ours is already written — edit it rather than regenerating.

The habit that matters most: **plan before you code**. Reviewing a plan takes seconds. Reviewing a wrong diff takes an hour, and you will be tempted to accept it because it is already written.

Claude Code — Power Features

These turn Claude Code from a helper into part of the team's process. All six are checked into the repo, so everyone gets them automatically.

Feature	Where it lives	What it is for
Custom slash commands	<code>.claude/commands/<name>.md</code>	A prompt you reuse, as a Markdown file with frontmatter. <code>\$ARGUMENTS</code> is replaced by what you type. This is the cheapest way to make good practice the default — nobody has to remember the long prompt.
Skills	<code>.claude/skills/<name>/SKILL.md</code>	Packaged know-how Claude loads when a task matches the description. Ours holds the migration pattern and RLS rules, so a new module is written our way without anyone pasting a template.
Subagents	<code>.claude/agents/<name>.md</code>	A separate agent with its own tools and its own context window. Use for work that would otherwise flood the main conversation — reviews, sweeps, test writing. It reports back; your main session stays focused.
MCP servers	<code>.mcp.json</code> or <code>claude mcp add</code>	Connects Claude to outside systems — your Supabase project, GitHub, Sentry. Checked into the repo, so everyone gets the same connections.
Hooks	<code>.claude/settings.json</code>	Shell commands that fire on events. Example: run <code>pnpm lint</code> automatically after every file edit, so formatting is never a review comment.
Permissions	<code>.claude/settings.json</code>	Pre-approve safe commands so you are not clicking allow all day, and deny the dangerous ones outright. Ours allows <code>pnpm</code> and read-only git, and asks before commit, push or PR.

The point of all six is the same: make the good path the easy path. Nobody should have to remember a long prompt or a checklist — the repo should hand it to them.

Our Commands, Subagents and Skills

Already written and committed in `.claude/`. Use them from day one — they encode the rules so you do not have to hold them in your head.

Name	Type	What it does
<code>/spec</code>	command	Turns an issue into a reviewable spec — module, tables touched, acceptance criteria, offline behaviour, security, tests. Writes no code. Run this first, always.
<code>/implement</code>	command	Implements an agreed spec inside its module boundary. Refuses to touch another module's tables or reference a post-MVP module.
<code>/review</code>	command	Reviews your branch as a strict reviewer: boundary violations, scope creep, RLS gaps, secrets, missing tests, stale docs.
<code>/ship</code>	command	Runs every gate — typecheck, lint, test, review, security — then prepares the commit and PR body. Stops and asks before committing.
<code>module-guard</code>	subagent	Checks a diff for module-boundary violations and reports them. Never edits. Use it after any change touching the database.
<code>test-writer</code>	subagent	Writes the missing tests, especially the negative ones — proving the wrong role gets zero rows, not just that the right one works.
<code>doc-keeper</code>	subagent	Keeps FEATURES.md, DECISIONS.md and CLAUDE.md in step with the code. Runs on Haiku, so it is cheap to call often.
<code>alarsio-module</code>	skill	Loads automatically when you create a module or add a migration. Holds our migration naming, the RLS pattern, and the checklist before a PR.

A normal feature is: `/spec` → agree it → `/implement` → `module-guard` → `test-writer` → `/ship`. Six commands, and the process runs itself.

The Documents That Run The Project

Ten files. Each has one job and one moment you read it. Keeping them true is part of the work, not overhead.

File	When you read it	What it holds
CLAUDE.md	Everyone, always	Claude reads it automatically. Rules, stack, owners, conventions. The most important file in the repo.
docs/PRD.md	Before building a feature	What we are building and why. The P0 list.
docs/ARCHITECTURE.md	Before touching the database	Module boundaries, public surfaces, the five ways modules may talk.
docs/FEATURES.md	Every PR	Single source of scope. Who owns what, what is done. **CI fails if a migration lands without updating it.**
docs/DECISIONS.md	When you choose	Why we picked X over Y. Never delete a decision — supersede it.
docs/WORKFLOW.md	Day one, then when stuck	The ten steps. Branch and commit conventions.
docs/SECURITY.md	Every security review	The checklist. Run it, don't skim it.
docs/TESTING.md	Writing tests	What to test where, and the manual pass before a PR.
docs/IMPROVEMENTS.md	When tempted to copy v1	The nine things we are deliberately doing differently.
docs/ONBOARDING.md	Your first hour	Setup, step by step.

FEATURES.md is the one with teeth. CI fails any PR that adds a migration without updating it — because undocumented scope drift is exactly what went wrong last time.

Git, PRs and Review

Four people in one repo for eight weeks. These rules are what stop that becoming a merge problem.

The rules

Thing	Rule	Why
Branch	<code>feat/<module>-<thing></code> · <code>fix/</code> · <code>chore/</code> · <code>docs/</code>	Branch from main, keep it short-lived, rebase rather than merging main into it.
Commit	Imperative, one logical change	“Add geofence check to site check-in”. Say what; if it is not obvious, say why in the body. Never <code>git add .</code> without reading what you staged.
PR size	One module, one concern	If a reviewer cannot hold it in their head, it is too big. Split it.
Reviewer	Someone who does not own the module	This is deliberate. It spreads knowledge and catches assumptions the owner cannot see.
Merge	CI green, one approval, conversations resolved	Enforced by branch protection, so nobody has to police it.

What a reviewer asks

1. Does this do what the issue asked, and only that?
2. Does it write to a table its module does not own?
3. Does it reference a post-MVP module?
4. Is RLS on every new table and view?
5. Does a policy compare a role inline instead of calling a named predicate?
6. Is there a test for the failure case, not only the happy path?
7. Could I fix a bug in this in three months?

Approve, or request changes with a reason. “Looks good” without reading is worse than no review — it creates a record saying the code was checked when it was not.

Testing and Security Review

Step 5 and step 6 of the loop. Both are mandatory, and both have a checklist so they do not depend on how much you remember that day.

Security checklist

Area	What to check
Always	RLS on every new table and view · named predicates, never inline role checks · no service-role key anywhere · no secret in the diff · no PII in a log line
Data access	A supervisor reaches only assigned sites — tested, not assumed · a supervisor cannot read wages · only an owner reopens a locked period · views are <code>security_invoker</code> · changing an id in the request cannot reach another organisation
Input	Validated server-side, not only in the form · no string concatenation into SQL · uploads check type and size · money and hours are range-checked
Worker data	Consent recorded before PII is entered · deleting removes access, never history · location captured once on an explicit tap, never in the background
Offline	A replayed push creates no duplicate · a device cannot overwrite an approved day · the device id is not trusted as identity

Testing rules

Rule	Meaning
Every PR adds a test	or explains in the PR why it does not
Test RLS negatively	prove the wrong user gets zero rows
Test every offline path	capture offline, kill the app, reopen, sync
Test money by hand	a worked example calculated in the test itself
Test every constraint	write the statement that should fail, assert it does

Under RLS, an UPDATE with no matching policy affects zero rows and raises nothing. A test wrapped in try/catch therefore passes vacuously. Assert the row did not change.

CI/CD with GitHub Actions

CI is not paperwork. It is the thing that lets four people move fast without checking each other's work by hand.

Job	When it runs	What it does
<code>verify</code>	Every PR and push to main	Typecheck, lint, test, build — all through Turborepo, so only what changed is rebuilt. A red build does not get reviewed.
<code>scope-guard</code>	Every PR	Fails if a migration lands without a <code>FEATURES.md</code> update. This is exactly how scope drift got into the previous build unnoticed — now it cannot.
service-role check	Every PR	Greps our source directories for a service-role reference and fails if one is found.
<code>claude.yml</code>	When you mention @claude	The Claude GitHub Action. Set it up once with <code>/install-github-app</code> as a repo admin, then mention @claude in any issue or PR comment and it will work on it.

Branch protection on `main`

1. Require a pull request before merging
2. Require 1 approving review
3. Require status checks to pass
4. Require conversation resolution before merging
5. No direct pushes to main — for anyone

Why Turborepo matters here

Turbo caches task results and knows the dependency graph, so CI only rebuilds what actually changed. A one-line change to the mobile app does not rebuild the web app. As the repo grows this is the difference between a 40-second CI and a 6-minute one.

Who Builds What, Week by Week

Two modules each, assigned so nobody waits on anybody. The order is set by dependencies, not preference.

TWO MODULES EACH



EIGHT WEEKS



Order is set by dependencies, not preference. Nothing in weeks 3-6 starts before weeks 1-2 publish their interface.

Ownership

Person	Modules	Note
Manish Tiwari	Sites · Attendance & Site Presence	The geofence, the roster, corrections. The heaviest field logic.
Ishant Bhoyar	Identity & Access · Workforce	Org, roles, RLS predicates everything else depends on. Build first.
Nikhil Mehta	Offline Sync · Materials & Inventory	The outbox and <code>sync_push</code> . The hardest module, and it spans the whole project.
Anuradha Tiwari	Audit, Retention & Guards · Daily Reporting	The audit trigger every module writes through, then the day's record.

Definition of done

1. The acceptance criteria in the issue are all true
2. `typecheck`, `lint`, `test` pass locally and in CI
3. A test exists for the failure case, not only the happy path
4. It works with the network off, if it is a field feature
5. `/security-review` run and `docs/SECURITY.md` walked
6. A different person reviewed and approved it
7. `docs/FEATURES.md` row ticked in the same PR